

Two Control Apps: A Live Style-Transfer Webcam and a Stepper Motor Mixer

Introduction

Our team set out to build software that gives a person direct, live control over something that normally stays hidden inside code. We ended up creating two separate applications that share one idea: put fast, hands-on control behind a clean, simple panel. The first is a real-time style-transfer webcam that repaints your live video feed in different artistic styles. The second is a mixer-style control app that drives five stepper motors at once. One controls images and the other controls motion, but both are really about the same thing: making a system feel instant and easy to play with.

Problem Statement

A lot of interesting technology is powerful but hard to actually use. Applying an artistic style to an image or moving a set of motors usually means editing code and running it again every time you want to change something. We wanted to remove that barrier. Our goal was to design two apps where a user could see the result immediately and adjust it live, without touching any code. For the webcam app, the main challenge was doing this fast enough to keep up with live video. For the motor app, the challenge was building a clear control panel for five motors that could also be developed and tested even when the physical hardware was not available.

Background Information

The webcam app uses the idea of style transfer, where the look of one image is applied to another. We combined two kinds of styling: trained models that produce painted looks, and lighter image filters that produce effects like a pencil sketch or pixel art quickly. Running these fast enough for live video required us to think carefully about how the work is scheduled on the computer.

The motor app controls stepper motors, which move in small, precise steps rather than spinning freely. Each motor is driven through a small driver board connected to a microcontroller, and the app sends signals that tell each motor how far and how fast to move. Because the motors move in fixed steps, they are well suited to precise, repeatable control from a simple interface.

Design Process

We worked in small steps, getting a basic version running first and then improving it. For the webcam app, our first version processed each video frame one after another, which made the video slow and choppy. To fix this, we redesigned the app so the separate jobs of capturing the camera, applying the style, and drawing the screen all run at the same time instead of waiting on each other. This single change is what made the video feel instant. We then added the ability to switch styles live, adjust how strong a style looks, and change the processing quality on the fly so we could balance speed against sharpness.

For the motor app, we started by getting reliable communication between the computer and the motors, then built a control panel that gives each of the five motors its own color, slider, and dial so they are easy to tell apart at a glance. We added several ways to control them, including precise individual adjustment and quick presets for moving several at once. An important design decision was to make the app fully usable without the motors plugged in, so we could keep building and testing even when hardware was limited. We designed the layout to stay readable and responsive, and made repeated small adjustments until the controls felt natural to use.

Results

Both apps met our main goals. The style-transfer webcam runs live: a user can point the camera at their face, switch between painted styles, sketches, and pixel art instantly, and see the effect update smoothly in real time. Keeping the display responsive regardless of how heavy the styling

was turned out to be the key measure of success, and our redesigned version handled this well. The stepper mixer successfully controls five motors from one panel, with clear per-motor feedback and multiple control modes. Because it also runs without hardware attached, we were able to demonstrate and refine the full interface reliably. Both apps performed close to what we expected, and the biggest surprises were in how much work was needed to make them feel simple and instant rather than in the underlying features themselves.

Reflection and Lessons Learned

The clearest lesson was that the interface is the project. The technology underneath only matters if a person can actually reach in and control it, and making something feel smooth and effortless is much harder than it first appears. We also learned the value of building in small, testable steps, and of designing our software so it could still be developed when resources were limited. Finally, working on two projects with a shared theme showed us that good design ideas like clear feedback, live control, and a clean layout carry over well from one kind of project to a completely different one.